CHARTER

Identify documented and undocumented capabilities and areas of potential instability of the Projects API.

Build

runTodoManagerRestAPI-1.5.5.jar

Area

Typical Scenarios – main functions and capabilities of TODO

Environment

macOS, Terminal, curl

START

12:24PM 10/12/2026

TESTER

Olivia Cardillo 260253654 olivia.cardillo@mail.mcgill.ca

#DURATION

54 minutes

TEST NOTES

12:24 PM — Sent a GET request to /projects to retrieve all existing projects. Received status 200 OK and a JSON response containing one project with ID 1, title "Office Work," completed and active statuses set to false, an empty description, and two associated tasks (IDs 1 and 2). The API successfully returned the existing project information.

Observation: The completed and active fields were represented as strings ("false") rather than JSON Boolean values. We will investigate whether this matches the API documentation.

12:26 PM — Sent a POST request to `/projects` with a JSON body containing the title "University Assignment" and description "Complete ECSE 429 project." Received status 201 Created. The API successfully created the project, automatically generated ID 2, and returned the correct title and description. The completed and active statuses were both set to false by default.

Observation: The completed and active attributes were again represented as strings ("false") rather than JSON Boolean values. The response also included a Location header pointing to `projects/2`.

12:30 PM — Sent a GET request to `/projects/2` to retrieve the project previously created in Test 2. Received status 200 OK and a JSON response containing project ID 2, title "University Assignment," description "Complete ECSE 429 project," and completed and active statuses set to false. The project was successfully retrieved with the correct information.

Observation: The response was wrapped in a "projects" array, even though only one project was requested. This differs from the POST response, which returned the project object directly.

12:31 PM — Sent a GET request to `/projects/99999` to retrieve a project with a nonexistent ID. Received status 404 Not Found and a JSON response containing the error message "Could not find an instance with projects/99999". The API correctly identified that the requested project does not exist.

Observation: The API returned an appropriate HTTP status code and a descriptive error message identifying the nonexistent project. No unexpected behavior was observed.

12:32 PM — Sent a HEAD request to `/projects/2` to retrieve the HTTP headers for an existing project. Received status 200 OK with Content-Type application/json and no response body. The API successfully returned the headers without retrieving the project data.

Observation: The HEAD request returned the expected status code and omitted the response body. No unexpected behavior was observed.

12:33 PM — Sent a HEAD request to `/projects/99999` to retrieve the HTTP headers for a nonexistent project. Received status 404 Not Found with no response body. The API correctly identified that the requested project does not exist.

Observation: The HEAD request returned the same HTTP status code as the previous GET request for the nonexistent project. No unexpected behavior was observed.

12:36 PM — Sent a POST request to `/projects/2` to update the description of an existing project. Received status 200 OK and a JSON response showing the updated description, "Updated description for exploratory testing." The title, completed status, and active status remained unchanged.

Observation: The POST method appears to support partial updates, modifying only the attributes specified in the request while preserving the remaining project attributes.

12:36 PM — Sent a GET request to `/projects/2` to verify that the changes made using POST were saved. Received status 200 OK and a JSON response containing the updated description, "Updated description for exploratory testing." The title, completed status, and active status remained unchanged.

Observation: The updated description persisted, confirming that POST successfully modified the project without affecting its other attributes.

12:37 PM — Sent a PUT request to `/projects/2` to update only the title of an existing project to "Updated University Assignment." Received status 200 OK and a JSON response containing the updated title. However, the description, which previously contained "Updated description for exploratory testing," was reset to an empty string. The completed and active statuses remained false.

Observation: Unlike POST, which preserved attributes not included in the request, PUT appears to overwrite unspecified attributes with their default values. This is an important behavioral difference between the two update methods. Further testing is needed to determine whether this behavior matches the API documentation.

12:38 PM — Sent a GET request to `/projects/2` to verify the changes made using PUT. Received status 200 OK and a JSON response containing the updated title "Updated University Assignment" and an empty description. The completed and active statuses remained false.

Observation: The changes made using PUT persisted. This confirms that PUT resets unspecified attributes to their default values, unlike POST, which preserves them. This difference should be compared against the API documentation.

12:39 PM — Sent a POST request to `/projects/99999` to update the title of a nonexistent project. Received status 404 Not Found and a JSON response containing the error message "No such project entity instance with GUID or ID 99999 found". The API correctly rejected the update because the specified project does not exist.

Observation: The error message differs from the one returned by GET for a nonexistent project, although both operations correctly returned status 404. The error message formats should be compared with the API documentation.

12:40 PM — Sent a PUT request to `/projects/99999` to update the title of a nonexistent project. Received status 404 Not Found and a JSON response containing the error message "Invalid GUID for 99999 entity project". The API correctly rejected the update because the specified project does not exist.

Observation: The error message differs from the one returned by POST for the same nonexistent project ID. Both methods returned status 404, but they use different error messages. This inconsistency should be compared with the API documentation.

12:40 PM — Sent a DELETE request to `/projects/2` to remove the project created earlier in the session. Received status 200 OK with an empty response body. The API indicated that the deletion was successful.

Observation: The DELETE request returned no confirmation message in the response body, despite specifying Content-Type application/json. Further verification using GET is needed to confirm that the project was removed. This behavior should also be compared with the API documentation.

12:41 PM — Sent a GET request to `/projects/2` to verify that the project deleted in the previous test was successfully removed. Received status 404 Not Found and a JSON response containing the error message "Could not find an instance with projects/2".

Observation: The API correctly returned an error when attempting to retrieve the deleted project, confirming that the DELETE operation successfully removed Project 2.

12:42 PM — Sent a DELETE request to `/projects/2` to attempt to delete a project that had already been removed. Received status 404 Not Found and a JSON response containing the error message "Could not find any instances with projects/2".

Observation: The API correctly rejected the repeated deletion request. The error message differs slightly from the GET response for the same nonexistent project ("any instances" versus "an instance"), although both returned status 404.

12:43 PM — Sent a GET request to `/projects/1/tasks` to retrieve all tasks associated with Project 1 ("Office Work"). Received status 200 OK and a JSON response containing two tasks: ID 2 ("file paperwork") and ID 1 ("scan paperwork"). Both tasks had their doneStatus set to false and contained a tasksof relationship referencing Project 1.

Observation: The API successfully retrieved both tasks associated with the project, consistent with the task references returned in Test 1. Task 1 also contained an associated category with ID 1.

12:43 PM — Sent a GET request to `/projects/99999/tasks` to retrieve tasks associated with a nonexistent project. Unexpectedly, received status 200 OK and a JSON response containing two tasks: ID 2 ("file paperwork") and ID 1 ("scan paperwork"). Both tasks belong to Project 1, as confirmed in Test 16.

Observation — BUG-01: The API returned tasks associated with an existing project even though the specified project ID does not exist. The API should reject the invalid project ID rather than returning unrelated project data. This behavior is inconsistent with GET `/projects/99999`, which correctly returned 404 Not Found.

```
(base) oliviacardillo@Olivias-MacBook-Air-2 ~ % curl -i http://localhost:4567/pr
ojects/99999/tasks
HTTP/1.1 200 OK
Date: Mon, 12 Oct 2026 16:43:59 GMT
Content-Type: application/json
Transfer-Encoding: chunked
Server: Jetty(9.4.z-SNAPSHOT)

{"todos":[{"id":"2","title":"file paperwork","doneStatus":"false","description":
"","tasksof":[{"id":"1"}]},{"id":"1","title":"scan paperwork","doneStatus":"fals
e","description":"","categories":[{"id":"1"}],"tasksof":[{"id":"1"}]}]}%
(base) oliviacardillo@Olivias-MacBook-Air-2 ~ %
```

12:45 PM — Sent a HEAD request to `/projects/99999/tasks` to retrieve the HTTP headers for tasks associated with a nonexistent project. Received status 200 OK with no response body, even though Project 99999 does not exist.

Observation — BUG-02: The API returned status 200 OK instead of the expected 404 Not Found for a nonexistent project. This is consistent with the unexpected behavior observed in Test 17, where GET `/projects/99999/tasks` returned tasks belonging to Project 1. Both operations appear to accept an invalid project ID without properly validating its existence. This behavior should be compared with the Swagger documentation.

```
(base) oliviacardillo@Olivias-MacBook-Air-2 ~ % curl -I http://localhost:4567/pr
ojects/99999/tasks
HTTP/1.1 200 OK
Date: Mon, 12 Oct 2026 16:45:18 GMT
Content-Type: application/json
Transfer-Encoding: chunked
Server: Jetty(9.4.z-SNAPSHOT)
```

12:46 PM — Sent a HEAD request to `/projects/1/tasks` to retrieve the HTTP headers for tasks associated with an existing project. Received status 200 OK with Content-Type application/json and no response body.

Observation: The API successfully returned the expected headers for an existing project. However, this response is identical to the HEAD response for the nonexistent Project 99999 in Test 18, further highlighting the lack of project ID validation in the tasks endpoint.

12:47 PM — Sent a POST request to `/projects` to create a new project for testing relationships between projects and tasks. Received status 201 Created and a JSON response containing the newly generated project ID 3, title "Relationship Testing Project," and description "Testing project-task relationships." The completed and active statuses were both set to false by default.

Observation: The API successfully created the project with an automatically generated ID and consistent default values. No task relationships were included in the creation response.

12:48 PM — Sent a GET request to `/projects/3/tasks` to retrieve tasks associated with the newly created Project 3. Received status 200 OK and a JSON response containing an empty todos array.

Observation: The API correctly returned an empty list for an existing project with no associated tasks. This differs from the behavior observed in Test 17, where a nonexistent project returned tasks belonging to another project.

12:49 PM — Sent a POST request to `/projects/3/tasks` with a JSON body containing Task ID 1 to create a relationship between Project 3 ("Relationship Testing Project") and the existing Task 1 ("scan paperwork"). Received status 201 Created with an empty response body.

Observation: The API indicated that the project-task relationship was successfully created but did not return any confirmation data in the response body. A GET request will be performed to verify that the relationship was established correctly.

12:50 PM — Sent a GET request to `/projects/3/tasks` to verify that Task 1 was successfully associated with Project 3. Received status 200 OK and a JSON response containing Task 1 ("scan paperwork"). The tasksof attribute referenced both Project 1 and Project 3.

Observation: The project-task relationship was successfully created. The task retained its original association with Project 1 while also being associated with Project 3, demonstrating that a task can belong to multiple projects simultaneously.

12:51 PM — Sent a GET request to `/todos/1` to verify the project-task relationship from the Todo side. Received status 200 OK and a JSON response containing Task 1 ("scan paperwork"), with the tasksof attribute referencing both Project 1 and Project 3.

Observation: The relationship was successfully maintained in both directions. Creating the relationship through the Projects API was also reflected when retrieving the task through the Todos API, confirming bidirectional consistency for this operation.

12:51 PM — Sent a DELETE request to `/projects/3/tasks/1` to remove the relationship between Project 3 ("Relationship Testing Project") and Task 1 ("scan paperwork"). Received status 200 OK with an empty response body.

Observation: The API indicated that the relationship was successfully removed. However, the response did not provide confirmation data. Further GET requests are needed to verify that the relationship was removed without deleting Task 1 or affecting its original association with Project 1.

12:52 PM — Sent a GET request to `/projects/3/tasks` to verify that the relationship between Project 3 and Task 1 was successfully removed. Received status 200 OK and a JSON response containing an empty todos array.

Observation: The task was successfully removed from Project 3, confirming that the DELETE operation removed the project-task relationship. Further verification is needed to ensure that Task 1 still exists and retains its original relationship with Project 1.

12:53 PM — Sent a GET request to `/todos/1` to verify that removing the relationship between Project 3 and Task 1 did not affect the original task. Received status 200 OK and a JSON response containing Task 1 ("scan paperwork"), with the tasksof attribute referencing only Project 1.

Observation: The DELETE operation successfully removed the relationship with Project 3 while preserving Task 1 and its original association with Project 1. No unexpected side effects were observed.

12:53 PM — Sent a GET request to `/projects/3/categories` to retrieve categories associated with Project 3 ("Relationship Testing Project"). Received status 200 OK and a JSON response containing an empty categories array.

Observation: The API correctly returned an empty list for an existing project with no associated categories. No unexpected behavior was observed.

12:54 PM — Sent a GET request to `/projects/99999/categories` to retrieve categories associated with a nonexistent project. Received status 200 OK and a JSON response containing an empty categories array, despite Project 99999 not existing.

Observation — BUG-03: The API returned status 200 OK instead of the expected 404 Not Found for a nonexistent project. This is inconsistent with GET `/projects/99999`, which correctly returned 404 Not Found. The API appears to treat a nonexistent project as though it were an existing project with no associated categories. This behavior should be compared with the Swagger documentation.

```
(base) oliviacardillo@Olivias-MacBook-Air-2 ~ % curl -i http://localhost:4567/pr
ojects/99999/categories
HTTP/1.1 200 OK
Date: Mon, 12 Oct 2026 16:54:24 GMT
Content-Type: application/json
Transfer-Encoding: chunked
Server: Jetty(9.4.z-SNAPSHOT)

{"categories":[]}%
```

12:55 PM — Sent a HEAD request to `/projects/99999/categories` to retrieve the HTTP headers for categories associated with a nonexistent project. Received status 200 OK with no response body, even though Project 99999 does not exist.

Observation — BUG-04: The API returned status 200 OK instead of the expected 404 Not Found for a nonexistent project. This is consistent with Test 29, where GET `/projects/99999/categories` also returned 200 OK. Both requests appear to accept a nonexistent project ID without validating its existence. This behavior should be compared with the Swagger documentation.

```
(base) oliviacardillo@Olivias-MacBook-Air-2 ~ % curl -I http://localhost:4567/pr
ojects/99999/categories
HTTP/1.1 200 OK
Date: Mon, 12 Oct 2026 16:55:05 GMT
Content-Type: application/json
Transfer-Encoding: chunked
Server: Jetty(9.4.z-SNAPSHOT)
```

12:55 PM — Sent a POST request to `/projects/3/categories` with a JSON body containing Category ID 1 to create a relationship between Project 3 ("Relationship Testing Project") and the existing Category 1. Received status 201 Created with an empty response body.

Observation: The API indicated that the project-category relationship was successfully created but did not return any confirmation data in the response body. A GET request will be performed to verify that the relationship was established correctly.

1:00 PM — Sent a GET request to `/projects/3/categories` to verify that Category 1 was successfully associated with Project 3. Received status 200 OK and a JSON response containing Category 1, titled "Office," with an empty description.

Observation: The project-category relationship was successfully created and persisted, confirming that the POST request in Test 31 worked as intended. No unexpected behavior was observed.

1:01 PM — Sent a DELETE request to `/projects/3/categories/1` to remove the relationship between Project 3 ("Relationship Testing Project") and Category 1 ("Office"). Received status 200 OK with an empty response body.

Observation: The API indicated that the project-category relationship was successfully removed but did not return any confirmation data. A GET request will be performed to verify that the relationship was deleted without affecting the original category.

1:02 PM — Sent a DELETE request to `/projects/3` to remove the temporary "Relationship Testing Project" created earlier in the session. Received status 200 OK with an empty response body.

Observation: The API indicated that the project was successfully deleted. A GET request will be performed to verify that Project 3 no longer exists.

1:03 PM — Sent a GET request to `/projects/3` to verify that the temporary project deleted in Test 34 was successfully removed. Received status 404 Not Found and a JSON response containing the error message "Could not find an instance with projects/3".

Observation: The API correctly rejected the request for the deleted project, confirming that the DELETE operation successfully removed Project 3.

1:04 PM — Sent a GET request to `/projects?title=Office%20Work` to investigate whether the Projects API supports filtering by title. Received status 200 OK and a JSON response containing Project 1 ("Office Work"), which matches the requested title.

Observation: The API returned a project matching the specified query parameter. However, since only one project currently exists, further testing with a nonmatching title is needed to determine whether the API actually filters the results.

1:04 PM — Sent a GET request to `/projects?title=NonexistentProject429` to test filtering by a title that does not exist. Received status 200 OK and a JSON response containing an empty projects array.

Observation: The API correctly returned no projects when the specified title did not match any existing project. Combined with Test 36, this confirms that the Projects API supports filtering by title.

1:05 PM — Sent a POST request to `/projects` with a JSON body containing only a description, omitting the title attribute. Received status 201 Created, and the API successfully created Project 4 with an empty title, the specified description, and completed and active statuses set to false.

Observation: The API allows projects to be created without specifying a title and automatically assigns an empty string as the default value. This suggests that the title attribute is not required by the implementation. Further investigation is needed to determine whether this behavior matches the API documentation.

1:05 PM — Sent a POST request to `/projects` with malformed JSON containing an extra comma after the description attribute. Received status 400 Bad Request and an error message indicating a MalformedJsonException. The API correctly rejected the invalid JSON input.

Observation: Unlike previous error responses, which used Content-Type application/json, this response specified text/html;charset=utf-8 despite containing a JSON-formatted error message. The error also exposed the internal Gson parser exception. This inconsistency should be compared with the API documentation to determine whether it represents a bug.

1:06 PM — Sent a GET request to `/projects?title=Malformed%20Project` to verify that the malformed JSON request from Test 39 did not create a project. Received status 200 OK and a JSON response containing an empty projects array.

Observation: No project matching the title "Malformed Project" was found, indicating that the malformed JSON request was rejected without creating the intended project. No unexpected behavior was observed.

1:07 PM — Sent a GET request to `/projects/4` to verify that the project created without a title in Test 38 was successfully removed. Received status 404 Not Found and a JSON response containing the error message "Could not find an instance with projects/4".

Observation: The API correctly returned an error when attempting to retrieve the deleted project, confirming that the DELETE operation successfully removed Project 4.

Additional Testing — XML Format

Time: 11 minutes

Additional exploratory testing was performed using XML to verify whether the Projects API supports XML requests and responses. The tests covered retrieving, creating, updating, and deleting projects, as well as handling invalid requests.

The results were consistent with the JSON testing. Additionally, the previously identified issue with GET `/projects/99999/tasks` was reproduced using XML, where tasks belonging to Project 1 were returned despite requesting a nonexistent project.

1:42 PM — Sent a GET request to `/projects` with the Accept header set to application/xml to retrieve all existing projects in XML format. Received status 200 OK and an XML response containing Project 1 ("Office Work"), with completed and active statuses set to false, an empty description, and two associated tasks (IDs 1 and 2).

Observation: The API successfully returned the existing project information in XML format. The data was consistent with the JSON response obtained during the previous session. The XML response represented the completed and active statuses as text values and used separate tasks elements for each associated task. No unexpected behavior was observed.

1:42 PM — Sent a GET request to `/projects/1` with the Accept header set to application/xml to retrieve a specific project in XML format. Received status 200 OK and an XML response containing Project 1 ("Office Work"), with completed and active statuses set to false, an empty description, and two associated tasks (IDs 1 and 2).

Observation: The API successfully retrieved the specified project in XML format. The returned information was consistent with the previous JSON response. Similar to JSON, the XML response was wrapped in a projects element despite requesting a single project. No unexpected behavior was observed.

1:43 PM — Sent a POST request to `/projects` with Content-Type and Accept headers set to application/xml. The XML body contained the title "XML Test Project" and description "Testing project creation using XML." Received status 201 Created and an XML response containing the newly generated project ID 5, the specified title and description, and completed and active statuses set to false.

Observation: The API successfully accepted an XML request body and returned the newly created project in XML format. The ID was automatically generated, and the completed and active attributes were assigned false by default, consistent with the behavior observed when creating projects using JSON. No unexpected behavior was observed.

1:44 PM — Sent a GET request to `/projects/5` with the Accept header set to application/xml to verify that the project created in Test 45 was successfully saved. Received status 200 OK and an XML response containing Project 5 ("XML Test Project"), with the correct description and completed and active statuses set to false.

Observation: The API successfully retrieved the newly created project in XML format. All attributes matched the values returned during creation, confirming that the XML POST request successfully saved the project.

1:45 PM — Sent a POST request to `/projects/5` with Content-Type and Accept headers set to application/xml to update only the description of an existing project. Received status 200 OK and an XML response containing the updated description, "Updated description using XML." The title, completed status, and active status remained unchanged.

Observation: The API successfully updated the specified attribute while preserving the remaining project attributes. This behavior is consistent with the partial update functionality observed when using POST with JSON.

1:47 PM — Sent a PUT request to `/projects/5` with Content-Type and Accept headers set to application/xml to update only the title of an existing project. Received status 200 OK and an XML response containing the updated title, "Updated XML Test Project." However, the description, which

previously contained "Updated description using XML," was reset to an empty value. The completed and active statuses remained false.

Observation: Unlike POST, which preserved unspecified attributes, PUT reset the description to its default empty value. This behavior is consistent with the results observed during JSON testing, suggesting that PUT replaces unspecified attributes with default values regardless of the request format.

1:48 PM — Sent a GET request to `/projects/5` with the Accept header set to application/xml to verify the changes made using PUT. Received status 200 OK and an XML response containing the updated title "Updated XML Test Project" and an empty description. The completed and active statuses remained false.

Observation: The changes made using PUT persisted when retrieving the project. This confirms that PUT reset the unspecified description to its default empty value, consistent with the behavior observed during JSON testing.

1:48 PM — Sent a GET request to `/projects/99999` with the Accept header set to application/xml to retrieve a nonexistent project. Received status 404 Not Found and an XML response containing the error message "Could not find an instance with projects/99999".

Observation: The API correctly rejected the request for a nonexistent project and returned the error message in XML format. The status code and error message were consistent with those observed during JSON testing. No unexpected behavior was observed.

1:49 PM — Sent a POST request to `/projects` with Content-Type and Accept headers set to application/xml. The XML body was intentionally malformed by omitting the closing description tag. Received status 400 Bad Request and an error message stating "Mismatched description and project at 99 [character 11 line 4]." The API correctly rejected the malformed XML request.

Observation: Although the error response was formatted as XML, the Content-Type header was text/html;charset=utf-8 instead of application/xml. This is consistent with the behavior observed when testing malformed JSON, where the API also returned a text/html Content-Type. This inconsistency should be compared with the API documentation.

1:50 PM — Sent a GET request to `/projects?title=Malformed%20XML%20Project` to verify that the malformed XML request from Test 51 did not create a project. Received status 200 OK and a JSON response containing an empty projects array.

Observation: No project matching the title "Malformed XML Project" was found, indicating that the malformed XML request was rejected without creating the intended project. No unexpected behavior was observed.

1:51 PM — Sent a GET request to `/projects/99999/tasks` with the Accept header set to application/xml to retrieve tasks associated with a nonexistent project. Unexpectedly, received status 200 OK and an XML response containing two tasks: ID 2 ("file paperwork") and ID 1 ("scan paperwork"). Both tasks were associated with Project 1.

Observation — BUG-05: The API returned tasks belonging to an existing project despite requesting tasks for a nonexistent project ID. This reproduces the unexpected behavior identified during JSON testing, confirming that the issue affects both response formats. The endpoint does not appear to validate the existence of the specified project before returning associated tasks.

```
(base) oliviacardillo@Olivias-MacBook-Air-2 ~ % curl -i -H "Accept: application/
xml" \
  http://localhost:4567/projects/99999/tasks
HTTP/1.1 200 OK
Date: Mon, 12 Oct 2026 17:51:17 GMT
Content-Type: application/xml
Transfer-Encoding: chunked
Server: Jetty(9.4.z-SNAPSHOT)

<todos><todo><doneStatus>false</doneStatus><description/><tasksof><id>1</id></ta
sksof><id>2</id><title>file paperwork</title></todo><todo><doneStatus>false</don
eStatus><description/><tasksof><id>1</id></tasksof><id>1</id><categories><id>1</
id></categories><title>scan paperwork</title></todo></todos>%
```

1:52 PM — Sent a DELETE request to `/projects/5` with the Accept header set to application/xml to remove the temporary project created during XML testing. Received status 200 OK with an empty response body.

Observation: The API indicated that the project was successfully deleted. Similar to the JSON DELETE operation, the XML response contained no confirmation message. A GET request will be performed to verify that Project 5 was removed.

1:52 PM — Sent a GET request to `/projects/5` with the Accept header set to application/xml to verify that the temporary project was successfully deleted. Received status 404 Not Found and an XML response containing the error message "Could not find an instance with projects/5".

Observation: The API correctly rejected the request for the deleted project, confirming that the XML DELETE operation successfully removed Project 5. The error response was also correctly formatted in XML, consistent with the requested Accept header.

** Ending session

Bugs Logged/Closed

(1) Performing GET `/projects/:id/tasks` with a non-existing Project ID (99999) returned tasks belonging to another project in the JSON response. This is incorrect behavior and was also reproduced using XML. ##BUG (Refer to BUG-01_GET_Tasks_Nonexistent_Project.docx)

(2) Performing HEAD `/projects/:id/tasks` with a non-existing Project ID (99999) returned status 200 OK instead of 404 Not Found. This is incorrect behavior. ##BUG (Refer to BUG-02_HEAD_Tasks_Nonexistent_Project.docx)

(3) Performing GET `/projects/:id/categories` with a non-existing Project ID (99999) returned status 200 OK and an empty categories list. This is incorrect behavior. ##BUG (Refer to BUG-03_GET_Categories_Nonexistent_Project.docx)

(4) Performing HEAD `/projects/:id/categories` with a non-existing Project ID (99999) returned status 200 OK instead of 404 Not Found. This is incorrect behavior. ##BUG (Refer to BUG-04_HEAD_Categories_Nonexistent_Project.docx)

Bugs Closed: None.

List of Testing Ideas

1. Compare each invalid-project-ID response against the Swagger documentation, including GET and HEAD requests for project tasks and categories.

2. Test the active and completed fields using true/false values, incorrect types and omitted attributes; verify stored values and response types.

3. Test POST versus PUT with all fields provided, blank values and omitted fields to determine exact replacement/update behavior.

4. Test title validation using an empty title, whitespace-only title, missing title and duplicate titles if applicable.

5. Test title filtering with multiple existing projects, multiple query parameters and partial versus exact matches.

6. Test project–task and project–category relationships using invalid task/category IDs, duplicate associations and repeated removal.

7. Confirm project–category DELETE via GET and verify that removing a relationship does not delete the category.

8. Test malformed payload handling and Content-Type / Accept negotiation across JSON and XML, including mismatched headers.

9. Investigate whether the XML project–task relationship endpoints behave consistently for valid IDs, not only invalid IDs.

Ares of Potential Risk

- Incorrect data exposure: BUG-01 returns tasks from Project 1 for a nonexistent project ID, which could lead a client to display unrelated information.

- Misleading success responses: BUG-02 through BUG-04 return 200 for invalid relationship URLs, potentially preventing clients from detecting invalid references.

- Unintended data loss: PUT clears unspecified fields; a client expecting a partial update could unintentionally remove previously saved information.
- Client integration problems: inconsistent error message wording, response wrappers and Content-Type headers may complicate response handling.
- Input validation uncertainty: the API accepts a project without a title, which could create incomplete records if a title is expected to be required.

Capabilities

- CRUD operations for Projects using JSON and XML (GET list/single, POST create/update, PUT update, DELETE).
- HEAD requests for existing and nonexistent individual project resources.
- Filtering projects by the title query parameter in JSON.
- Creating, querying and removing project–task relationships in JSON, with bidirectional relationship changes verified via /todos/1.
- Querying, creating and removing project–category relationships in JSON (relationship creation confirmed; deletion response observed but not independently verified by a subsequent GET).
- Accepting XML request bodies and returning XML responses for core Projects operations and expected 404 errors.
- Rejecting malformed JSON and XML payloads with HTTP 400 responses.
- Returning HTTP 404 for nonexistent individual projects and after confirmed project deletion.

Capability demonstration script: projectsCapabilities.sh